MODERN USEFUL APIS

@frontend_in_depth

WebGPU API

Browser-based GPU access for fast 3D graphics and computations. Beats WebGL in speed, native in Chrome 113+.

```
import { useEffect } from 'react';

function WebGPUCanvas() {
  useEffect(() => {
    async function init() {
      if (!navigator.gpu) return alert('WebGPU not supported!');
      const canvas = document.getElementById('gpu');
      const ctx = canvas.getContext('webgpu');
      const adapter = await navigator.gpu.requestAdapter();
      const device = await adapter.requestDevice();
      ctx.configure({ device, format: 'bgra8unorm' });
    }
    init();
  }, []);
  return <canvas id="gpu" />;
}
export default WebGPUCanvas;
```

WebAuthn API

Passwordless login with biometrics/security keys, browser-native, Chrome/Edge support.

```
function WebAuthnLogin() {
  const register = async () => {
    if (!window.PublicKeyCredential) return alert('WebAuthn not supported!');
    await navigator.credentials.create({
      publicKey: {
        challenge: new Uint8Array([1, 2, 3]),
        rp: { name: 'MyApp' },
        user: { id: new Uint8Array([1]), name: 'user@example.com' },
        pubKeyCredParams: [{ type: 'public-key', alg: -7 }]
      }
    });
  };
  return <button onClick={register}>Login</button>;
}
export default WebAuthnLogin;
```

CoinGecko API

Real-time crypto prices/market data, JSON, free, no key for basic calls.

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function CryptoPrices() {
  const [coins, setCoins] = useState([]);
  useEffect(() => {
    axios.get('https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&per_page=3')
      .then(res => setCoins(res.data));
  }, []);
  return coins.map(coin => <div key={coin.id}>{coin.name}: ${coin.current_price}</div>);
}
export default CryptoPrices;
```

WebRTC API

Real-time video/audio/data sharing, browser-native, Chrome/Firefox/Edge support.

```
import { useRef, useEffect } from 'react';

function VideoCall() {
  const videoRef = useRef();
  useEffect(() => {
    navigator.mediaDevices.getUserMedia({ video: true })
      .then(stream => videoRef.current.srcObject = stream);
  }, []);
  return <video ref={videoRef} autoPlay />;
}
export default VideoCall;
```

WEB AUDIO API

The Web Audio API provides a powerful and versatile system for controlling audio on the Web, allowing developers to choose audio sources, add effects to audio, create audio visualizations, apply spatial effects (such as panning) and much more.

```
const audioContext = new (window.AudioContext ||
                                window.webkitAudioContext)();
const oscillator = audioContext.createOscillator();
oscillator.frequency.setValueAtTime(440, audioContext.currentTime);
oscillator.connect(audioContext.destination);
oscillator.start();
oscillator.stop(audioContext.currentTime + 1);
```

Open Trivia Database API

Free API with 3,000+ trivia questions across 9 categories, difficulty levels, and types (multiple choice/true false). JSON responses, no key needed, daily fresh content.

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function TriviaQuiz() {
  const [question, setQuestion] = useState('');
  useEffect(() => {
    axios.get('https://opentdb.com/api.php?amount=1')
      .then(res => setQuestion(res.data.results[0].question));
  }, []);
  return <div dangerouslySetInnerHTML={{ __html: question }} />;
}
export default TriviaQuiz;
```

WEB INDEXDB API

IndexedDB is a low-level API for client-side storage of significant amounts of structured data, including files/blobs. This API uses indexes to enable high-performance searches of this data. While Web Storage is useful for storing smaller amounts of data, it is less useful for storing larger amounts of structured data.

```
// Open (or create) the database
const dbName = "InstagramPostsDB";
const dbVersion = 1;

const request = indexedDB.open(dbName, dbVersion);
//handles errors that may occur during the database opening process.
request.onerror(() => {});
//specifies the actions to be taken when the database structure is being upgraded.
request.onupgradeneeded(() => {});
//defines the actions to be taken upon successful opening of the database.
request.onsuccess(() => {});
```

Spoonacular API

Recipe search with 365k+ recipes, ingredients, nutrition info. JSON for meal plans, random suggestions; free tier with key.

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function RecipeSearch() {
  const [recipes, setRecipes] = useState([]);
  useEffect(() => {
    axios.get('https://api.spoonacular.com/recipes/random?number=3&apiKey=YOUR_KEY')
      .then(res => setRecipes(res.data.recipes));
  }, []);
  return recipes.map(r => <div key={r.id}>{r.title}</div>);
}
export default RecipeSearch;
```

WEB NOTIFICATION API

The Notification interface of the Notifications API is used to configure and display desktop notifications to the user.

```
Notification.requestPermission()
    .then( permission => {
      new Notification('Hello, World!');
    });
```

WEB WORKERS API

Web Workers makes it possible to run a script operation in a background thread separate from the main execution thread of a web application.

```
const worker = new Worker('worker.js');
worker.postMessage('Hello from main script!');
```

WEB POINTER LOCK API

The Pointer Lock API (formerly called Mouse Lock API) provides input methods based on the movement of the mouse over time (i.e., deltas), not just the absolute position of the mouse cursor in the viewport. It gives you access to raw mouse movement, locks the target of mouse events to a single element

```
const element = document.getElementById('yourElementId');
element.requestPointerLock();
```

GAMEPAD API

The Gamepad API is a way for developers to access and respond to signals from gamepads and other game controllers in a simple, consistent way.

```
window.addEventListener("gamepadconnected", (event) =>
    console.log("Gamepad connected:", event.gamepad.id)
);
window.addEventListener("gamepaddisconnected", (event) =>
    console.log("Gamepad disconnected:", event.gamepad.id)
);
```

WEB PAYMENT REQUEST API

The Payment Request API provides a consistent user experience for merchants and users. It is not a new way of paying for things; instead, it's a way for users to select their preferred way of paying for things and make that information available to a merchant.

```
const supportedInstruments = [{ supportedMethods: 'basic-card' }];
const paymentDetails = { total:
    { label: 'Total', amount: { currency: 'USD', value: '10.00' } } };
const paymentPromise = new PaymentRequest(supportedInstruments, paymentDetails);
paymentPromise.show().then(paymentResponse =>
      paymentResponse.complete('success')
  );
```